

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ESPECIALIDAD DE INGENIERÍA INFORMÁTICA



Proyecto de Diseño y Desarrollo de Software [1INF54-0982]

Estándar de Contenerización con Docker

Equipo: 6F

Integrantes:

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

Historial de versiones

Fecha	Versión	Descripción	Autor
15/09/2026	1.0	Se ha redactado la primera versión del estándar de contenerización.	Equipo 6F

Contenido

1. Introducción	4
2. Buenas prácticas para el Dockerfile	4
2.1. Imágenes base	4
2.2. Builds multi-etapa	4
2.3. Optimización de capas	5
2.4. Compatibilidad de scripts y finales de línea	5
3. docker-compose.yml	5
3.1. Gestión de recursos y límites	6
3.2. Aislamiento de redes internas	6
3.3. Políticas de rotación de logs	7
4. Gestión segura de variables de entorno	7
5. Mapeo estandarizado de puertos y volúmenes	7
5.1. Puertos	7
5.2. Volúmenes	8
6. Referencias y recursos	8

1. Introducción

Este documento define las buenas prácticas para la contenerización de los distintos servicios del sistema SysMile (backend Java, frontend web y base de datos PostgreSQL), abarcando la redacción de los archivos Dockerfile, la orquestación local mediante docker-compose.yml, la gestión de variables de entorno sensibles y el mapeo estandarizado de puertos y volúmenes.

2. Buenas prácticas para el Dockerfile

2.1. Imágenes base

- Usar imágenes base ligeras basadas en Alpine Linux cuando sea posible (eclipse-temurin:21-jre-alpine para el backend, node:20-alpine para el proceso de build del frontend), reduciendo el tamaño final de la imagen y la superficie de ataque.
- Fijar siempre una versión específica de la imagen base (no usar la etiqueta latest) para garantizar builds reproducibles.

2.2. Builds multi-etapa

- Usar builds multi-etapa (multi-stage builds) tanto en el backend como en el frontend: una etapa de compilación con las herramientas de build (Maven, Node) y una etapa final mínima que solo contiene el artefacto compilado.

None

```
# Etapa de build
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

# Etapa final
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/sysmile-backend.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2.3. Optimización de capas

- Copiar primero los archivos de definición de dependencias (pom.xml, package.json) e instalar dependencias antes de copiar el código fuente, aprovechando el cacheo de capas de Docker.
- Combinar instrucciones RUN relacionadas en una sola línea (usando && y \) para reducir el número de capas intermedias.
- Incluir un archivo .dockerignore que excluya node_modules, target/, .git y archivos .env del contexto de build.
- Ejecutar el proceso de la aplicación con un usuario no root dentro del contenedor (USER appuser), nunca como root en producción.

2.4. Compatibilidad de scripts y finales de línea

- Para garantizar la correcta ejecución de scripts de inicio (entrypoints) en los contenedores basados en Alpine Linux, independientemente del sistema operativo del desarrollador, se debe configurar obligatoriamente un archivo .gitattributes en la raíz del repositorio.
- Este archivo debe contener la regla * text=auto eol=lf para forzar que los finales de línea de los archivos de texto se normalicen a LF (Line Feed). Esto previene errores fatales causados por retornos de carro (CRLF) invisibles al montar volúmenes o compilar imágenes desde distintos entornos de desarrollo.

3. docker-compose.yml

- Definir un servicio por cada componente del sistema: backend, frontend, db (PostgreSQL) y, de ser necesario, redis para caché.
- Nombrar los servicios en minúsculas y con guiones si son compuestos (route-optimizer si se separa como servicio independiente).
- Declarar explícitamente las dependencias entre servicios con depends_on, y usar condiciones de healthcheck para que el backend espere a que la base de datos esté realmente lista, no solo a que el contenedor haya iniciado.

```
JSON
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
```

```
    interval: 5s
    timeout: 5s
    retries: 5

backend:
  build: ./backend
  depends_on:
    db:
      condition: service_healthy
  env_file: ./backend/.env
  ports:
    - "8080:8080"

frontend:
  build: ./frontend
  ports:
    - "5173:80"
  depends_on:
    - backend
```

3.1. Gestión de recursos y límites

Para prevenir que la ejecución intensiva de los algoritmos de optimización (ALNS/IPSO) agote los recursos del sistema anfitrión, es obligatorio definir límites de hardware para cada contenedor.

Se debe utilizar el bloque `deploy.resources` en el archivo `docker-compose.yml` para establecer límites (limits) y reservas (reservations) explícitos de memoria RAM y CPU. El contenedor del backend debe tener un límite estricto de memoria que concuerde armónicamente con los parámetros de configuración de la máquina virtual de Java (JVM).

3.2. Aislamiento de redes internas

Queda desaconsejado depender de la red puente por defecto (default bridge) de Docker Compose. Se debe declarar explícitamente una red interna personalizada en el bloque `networks` (por ejemplo, `sysmile_network`) y acoplar todos los servicios del proyecto a ella.

Esta práctica previene conflictos de resolución DNS y colisiones de IP cuando el desarrollador ejecuta múltiples proyectos o entornos aislados en la misma máquina anfitriona.

3.3. Políticas de rotación de logs

Para evitar la saturación progresiva del almacenamiento en disco debido a la acumulación indefinida de registros de consola por parte del motor de Docker, todo servicio transaccional debe incluir una política estricta de rotación de logs.

Se debe configurar el bloque logging utilizando el controlador estándar json-file con opciones paramétricas definidas, aplicando límites como max-size: "10m" and max-file: "3" para controlar el volumen histórico retenido por contenedor.

4. Gestión segura de variables de entorno

- Nunca incluir credenciales de base de datos, claves JWT ni secretos de terceros directamente en el docker-compose.yml ni en el Dockerfile; deben provenir de archivos .env excluidos del control de versiones.
- Mantener un archivo .env.example en el repositorio con todas las variables requeridas y valores de ejemplo no sensibles, para que cualquier desarrollador pueda configurar su entorno local.
- En entornos de staging/producción, las variables sensibles se inyectan mediante el gestor de secretos de la plataforma de despliegue (no mediante archivos .env versionados), siguiendo el principio de menor privilegio.
- No exponer variables de entorno sensibles como argumentos de build (--build-arg) ya que quedan registradas en el historial de capas de la imagen; usarlas únicamente como variables de entorno en tiempo de ejecución o mediante Docker secrets.

5. Mapeo estandarizado de puertos y volúmenes

5.1. Puertos

Servicio	Puerto interno	Puerto expuesto (local)
backend	8080	8080
frontend	80	5173
db (PostgreSQL)	5432	5432
redis (opcional)	6379	6379

- En entornos de staging/producción, los puertos internos no se exponen directamente al host; el acceso se realiza a través de un proxy inverso (Nginx) o del balanceador de carga de la plataforma.

5.2. Volúmenes

- Persistir los datos de PostgreSQL mediante un volumen nombrado (sysmile_db_data), nunca en un volumen anónimo, para evitar pérdida de datos al recrear contenedores.
- Los volúmenes de desarrollo que montan el código fuente para hot-reload (bind mounts) se definen únicamente en un docker-compose.override.yml de uso local, nunca en el docker-compose.yml base usado en CI/CD.
- Nombrar los volúmenes con el prefijo del proyecto (sysmile_) seguido del propósito (sysmile_db_data, sysmile_redis_data).

6. Referencias y recursos

Documentación oficial de Docker: <https://docs.docker.com/>

Docker — Best practices for writing Dockerfiles:
<https://docs.docker.com/build/building/best-practices/>

Documentación de Docker Compose: <https://docs.docker.com/compose/>